



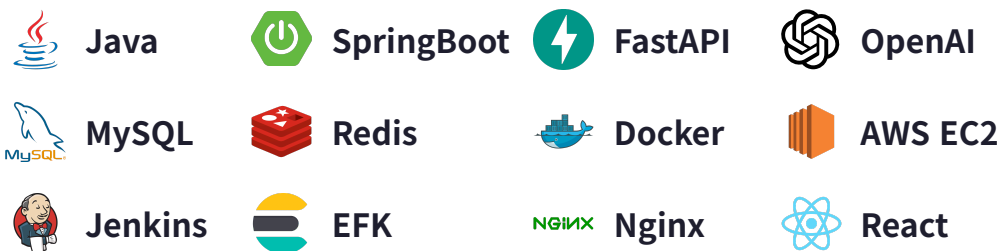
서비스

운세 강화게임이 결합된 데일리 운세 플랫폼 자율 PJT 우수상 2등

AI 기반 운세 계산·강화·공유 + 포춘쿠키 수집·랭킹 시스템을 결합한 운세 강화 게임 & 소셜 웹 서비스

[GitHub Repository](#)

기술 스택



역할 — Backend

인증/보안

OAuth2 + JWT, 토큰 회전(RTR) 및 CORS/SSL 최적화

랭킹 설계

Redis ZSET(실시간) + MySQL 스냅샷(영속성) 랭킹 이원화 구축

게임 로직

먹등성 키를 활용한 일일 제한(Cap) 및 동시성 제어

API 설계

공유 토큰 기반 딥링크 API 및 쿠키/도감 도메인 구현

협업 표준화

Swagger 기반 API 명세 및 DTO/에러 스펙 표준화

작업 기간

2025. 10. 13 - 2025. 11. 20 (6주)

SSAFY 13기 자율 프로젝트

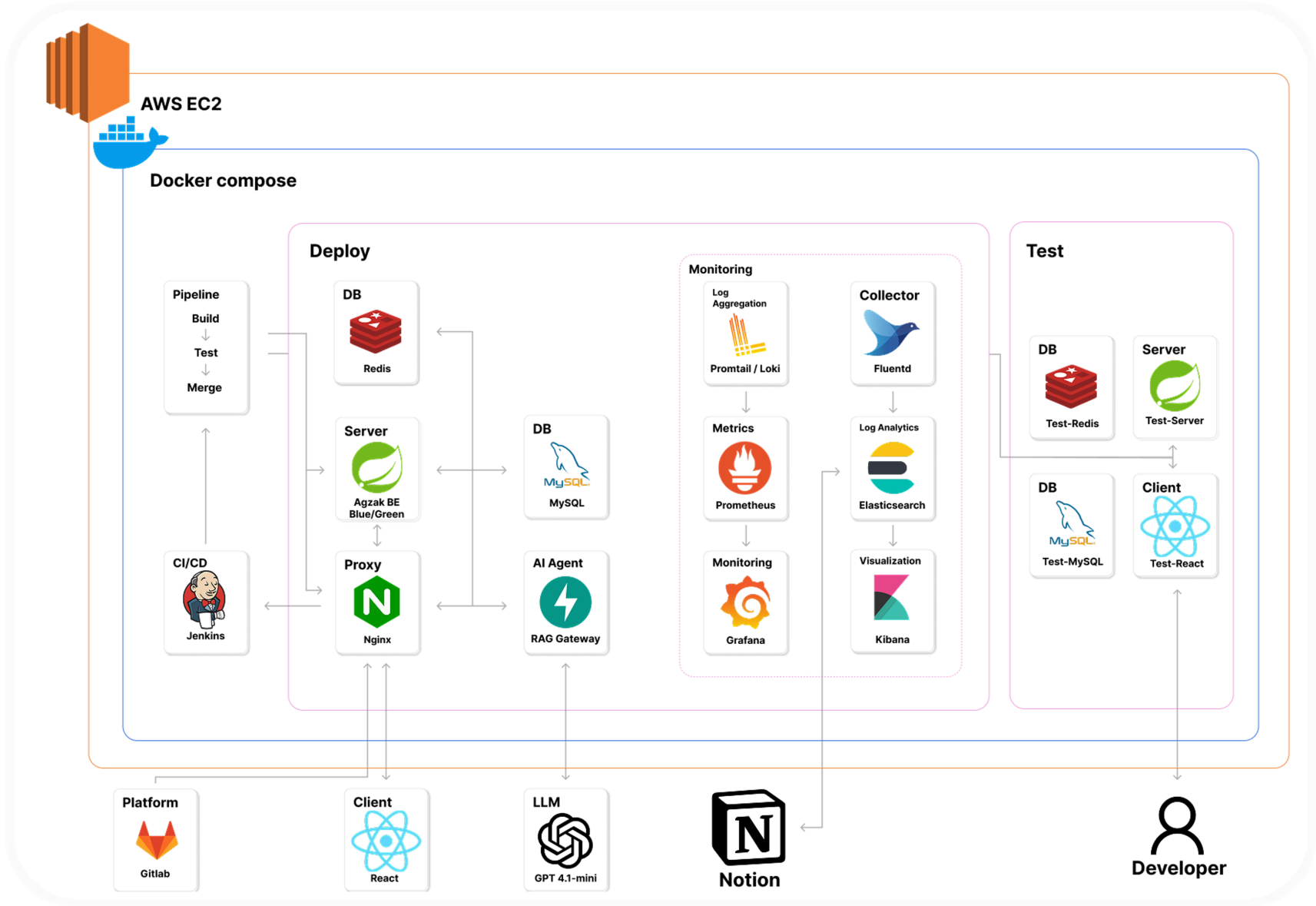
구성원

총 6명

FE 3명 · BE 2명 (본인포함) · Infra 1명

프로젝트 회고

- 실 트래픽 운영을 처음 받아본 경험 — 14일간 1,100명+ / 11만 이벤트
 - 수치로 안전했음을 증명할 수 있는 구조의 가치를 체감
- 환경이 만드는 장애를 처음 진단 — 스케줄러 데드락
 - 다중 인스턴스 + 자정 cron = 데드락
 - ShedLock + 매시간 분할로 발생률 0건 / 다음엔 예방적으로 잡기
- 같은 단어 안의 다른 요구를 분리 — 랭킹 이원화
 - Redis ZSET(실시간) + MySQL 스냅샷(영속)으로 Slow Query 0건
- 먹등성 = 비즈니스 정합성의 도구
 - 게임 일일 Cap에 먹등키 — 더블클릭·재시도에도 정확한 cap
- 방어를 코드로 녹였지만, 효과 증명은 숙제
 - JWT RTR + Redis 블랙리스트로 토큰 탈취 시나리오 방어
 - 다음엔 모의 침해 회귀 테스트로 효과 증명
- 모니터링은 질문하는 사람이 살아있게 한다
 - 대시보드를 매일 판독하며 "어떤 수치가 위험한가"를 묻는 습관 학습



① Deploy 환경

AWS EC2 · Blue/Green

- ▶ 헬스 엔드포인트(/actuator/health)로 트래픽 전환 트리거 제공
- ▶ ShedLock 으로 두 슬롯 동시 기동 안전성 보장 (스케줄러 중복 실행 차단)
- ▶ 운영/테스트 환경 동일 이미지 — 환경변수만으로 기동 분기
- ▶ Spring Boot 두 슬롯 + Nginx upstream swap 으로 무중단 배포

배포 다운타임 0초 · 동시 두 슬롯 안전

② CI / CD

Jenkins · 4 파이프라인

- ▶ Gradle 빌드 → Docker 이미지 → 레지스트리 푸시 자동화 흐름 호환
- ▶ health=UP 응답 보장으로 무중단 슬롯 컷오버
- ▶ 운영 동일 이미지로 테스트 환경(격리 DB/Redis) 회귀 검증 통과
- ▶ 4 파이프라인(운영·테스트 × BE·FE) + Mattermost 자동 알림

Docker 캐시 적중 시 빌드 1m 20s → 14s

③ 모니터링 · 로그

Prom · Graf · Loki · EFK

- ▶ Spring Actuator 메트릭 + trace_id 표준 로그 포맷 노출
- ▶ Mattermost 알람(5xx · Broken Pipe · Deadlock 40001) 추적·원인 규명
- ▶ 메트릭 — Prometheus + Grafana + Exporter (서버·MySQL·Nginx)
- ▶ 로그 — Loki/Promtail · EFK 병행 + Notion DB 일별 자동 리포트

14일 운영 Slow Query 0건 · ShedLock 적용 후 자정 데드락 재발 0건

④ AI 연동 · Test

FastAPI 게이트웨이 호출

- ▶ 운세 계산(결정론, Swissep) / 해설(AI 변동) 책임 분리 설계
- ▶ 항목별 AI 호출 병렬화 + 응답 개심으로 Broken Pipe 회복
- ▶ 외부 호출은 AI 게이트웨이 1곳으로 통제 — 내부 네트워크 호출만
- ▶ AI 게이트웨이 별도 컨테이너 (FastAPI · OpenAI · RAG · Swissep)

AI 호출 분리·병렬·개싱 → Broken Pipe 회복

PROBLEM → SOLUTION

"오늘 랭킹은 즉시 반영, 과거 랭킹은 그 시점 그대로 보존" — 두 요구는 **같은 자료구조로 풀 수 없다**. 갱신 빈도·정렬 비용·보존 요구가 다르므로 저장소 자체를 분리한다.

Before vs After

BEFORE

단일 MySQL + ORDER BY

- ✗ 매 요청 풀스캔 — 데이터 변경 없어도 매번 전체 read
- ✗ 세 가지 정렬 모두 메모리 — total / original / gain
- ✗ "내 등수" 추가 카운트 쿼리 — 결과를 다시 정렬
- ✗ 과거 일자 보존 X — Fortune 갱신 시 어제 순위 흔들림

AFTER

Redis ZSET + MySQL 스냅샷

- ✓ 점수 갱신 = ZADD $O(\log N)$ — DB 정렬보다 압도적
- ✓ TOP N = ZREVRANGE 단건 — 정렬 비용 0
- ✓ "내 등수" = ZREVRANK 한 번 — 별도 쿼리 X
- ✓ 과거 일자는 MySQL 스냅샷 영속 — 갱신에도 흔들리지 않음

동기화 전략 — ShedLock 2 스케줄 + Fallback



```
domain/ranking/service/RankingService.java

@Scheduled(cron = "55 59 * * *", zone = "Asia/Seoul")
@SchedulerLock(name = "rankingSnapshot",
    lockAtMostFor = "PT4M", lockAtLeastFor = "PT20S")
public void todayRebuild() { rankingService.rebuildTodaySnapshot(); }

@Transactional
public void rebuildTodaySnapshot() {
    LocalDate today = redisStore.today();
    var all = redisStore.readTop(today, redisStore.zTotal(today),
        Integer.MAX_VALUE);

    // ★ 탈퇴 회원 prune - Redis 고아 데이터 정리
    Set<Long> ids = all.stream().map(UserRow::memberId).collect(toSet());
    Set<Long> alive = mysqlStore.existingMemberIds(ids);
    for (Long mid : ids)
        if (!alive.contains(mid)) redisStore.removeTodayUser(mid);

    // 살아있는 회원만 rank_no 1번부터 영속 저장
    int rank = 1;
    List<Ranking> rows = new ArrayList<>(alive.size());
    for (var r : all) {
        if (!alive.contains(r.memberId())) continue;
        rows.add(Ranking.builder()
            .member(mysqlStore.refMember(r.memberId())) // em.getReference (N+1 회피)
            .date(today).rankNo(rank++)
            .totalScore(r.total()).originalTotalScore(r.original())
            .build());
    }
    mysqlStore.replaceTodaySnapshot(today, rows); // deleteByDate + saveAll
}

// ★ 위기 fallback - Redis 죽어도 MySQL 스냅샷으로 자동 대체
public RankingListDto getTodayTopN(int limit) {
    try { /* Redis 경로 */ }
    catch (Exception e) { return getSnapshotTopN(d, limit); }
}
```

0건

Slow Query

14일 운영 · MySQL QPS 한 자리수 유지

$O(\log N)$

랭킹 갱신·조회

ZADD · ZREVRANGE · ZREVRANK

0초

캐시 TTL 대기

강화 즉시 랭킹 반영

3축

단일 장애점 회피

Fallback · Reseed · Prune

1.1천 명

활성 사용자

14일 (2025.11.11~24) · 신규 97%

11만

이벤트 수

강화 시도 / 운세 조회 / 쿠키 굶기

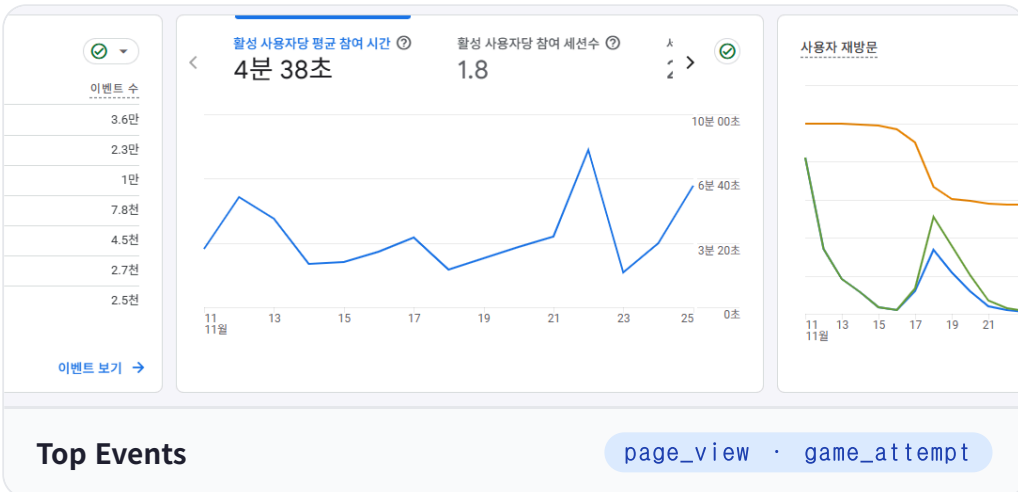
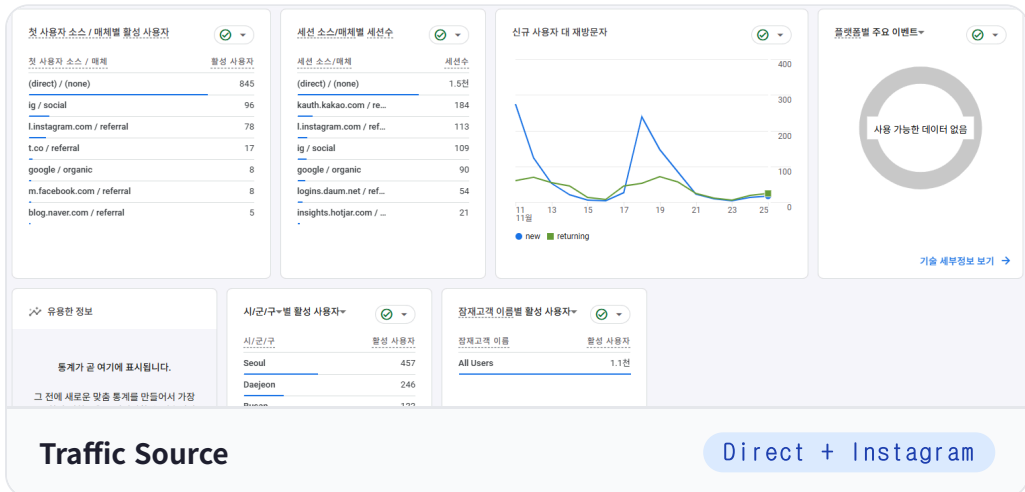
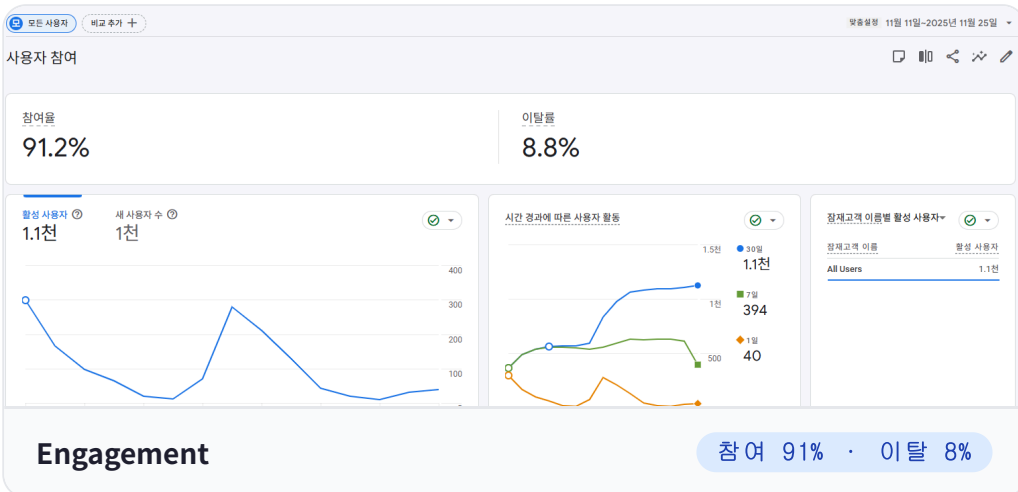
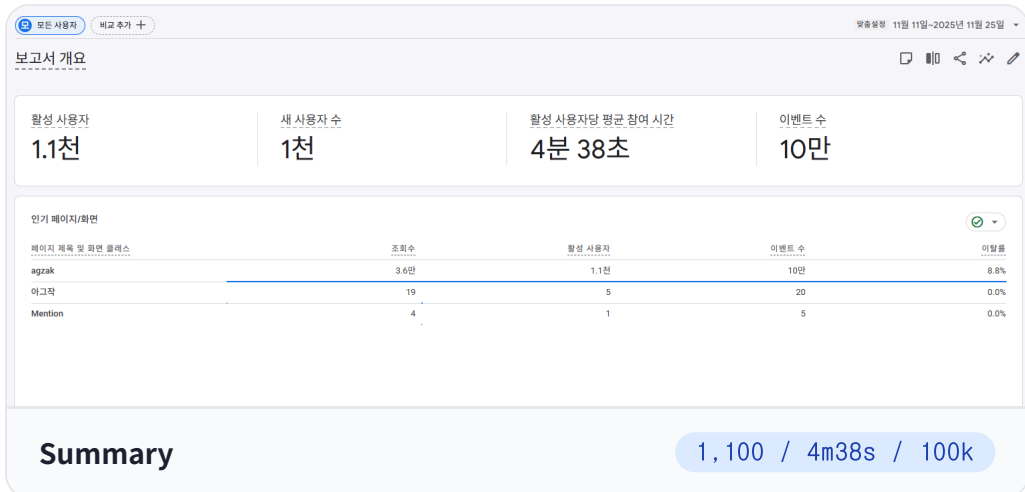
91 %

참여율 (이탈 8%)

GA4 — agzak 페이지 8.8% 이탈

Google Analytics 4 — 14일 운영 데이터

2025.11.11 ~ 11.24



구조적 장애 해결 — Before vs After

velog 트러블슈팅 2건

① 심야 스케줄러 데드락

14일 운영 중 23:59 cron + Blue/Green 두 컨테이너 동시 트리거로 InnoDB Deadlock(SQLState 40001) 발생. Mattermost 5xx FIRING 알람으로 발견 → 로그 추적 후 ShedLock + 매시간 분할로 단일 인스턴스 실행 보장.

	BEFORE	AFTER (SHEDLOCK 적용)
자정 데드락 알람	매일 1회	재발 0건
랭킹 스냅샷 누락	발견됨	0건
스케줄러 중복 실행	두 컨테이너 모두	한쪽만 + 즉시 skip

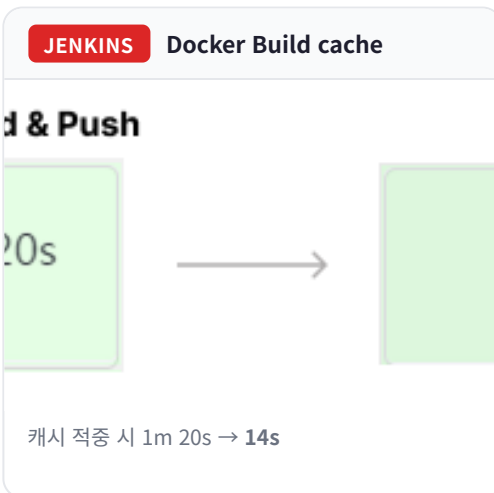
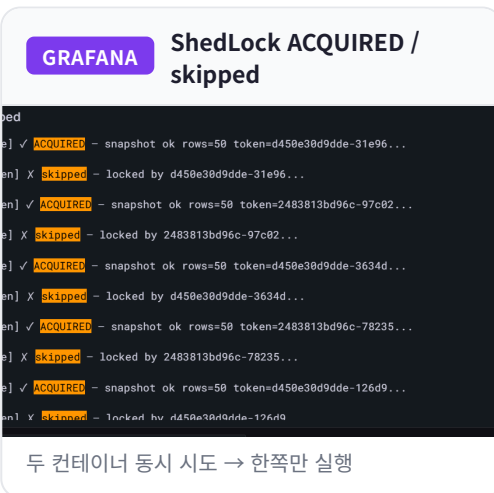
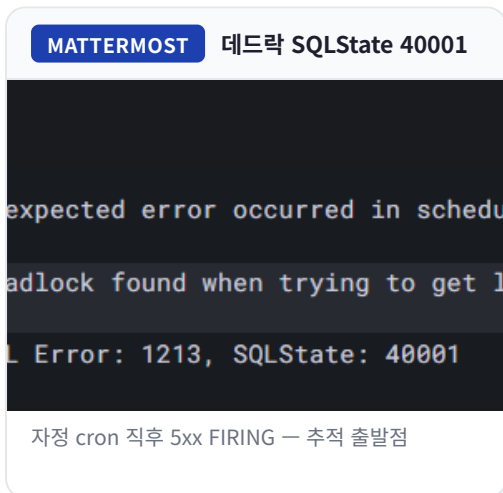
② 랭킹 저장소 이원화

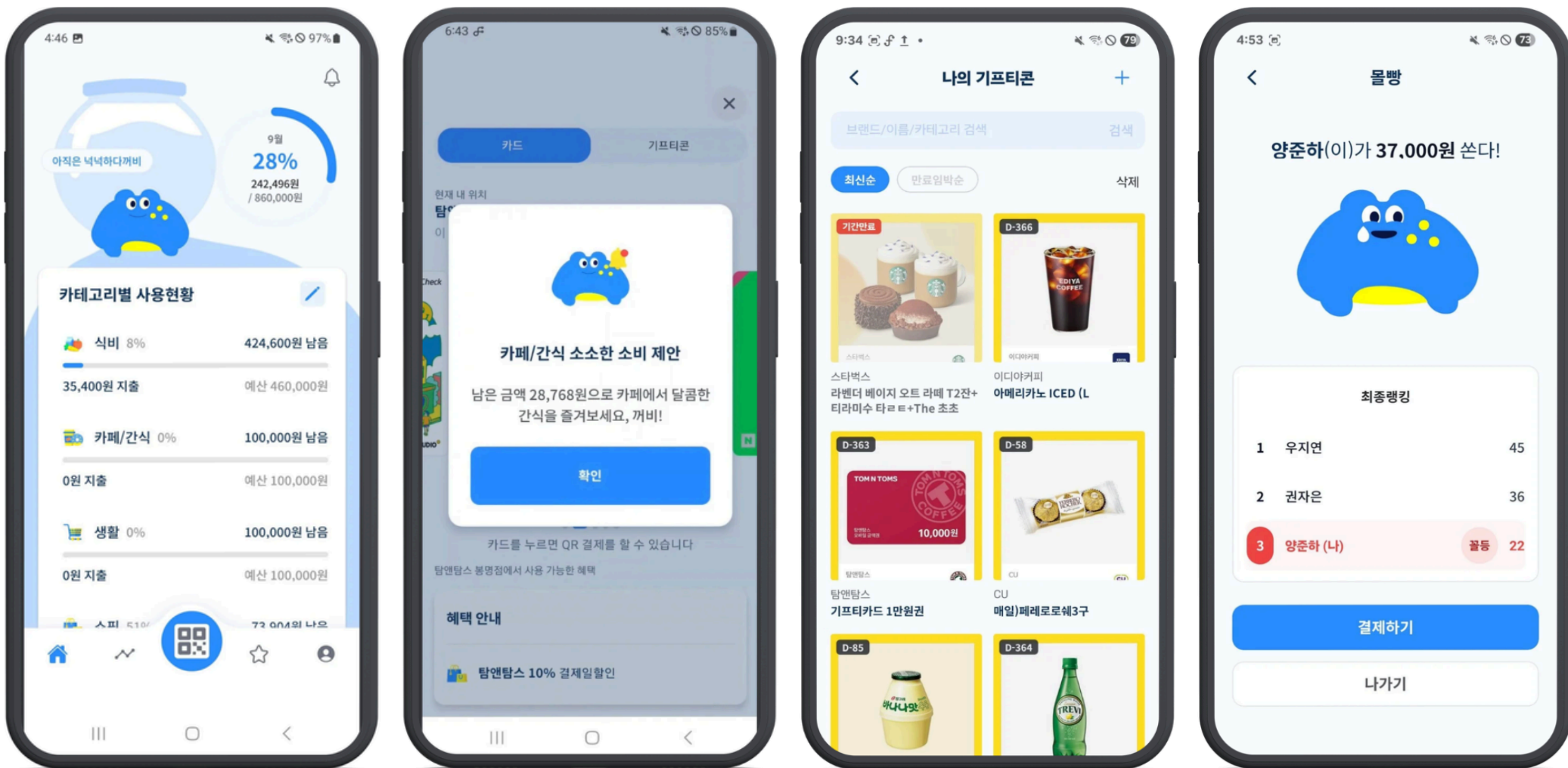
단일 MySQL ORDER BY 로는 실시간성·영속성·과거 보존 세 요구를 만족시킬 수 없음. Redis ZSET(실시간) + MySQL 스냅샷(영속) 으로 저장소 자체를 분리.

	BEFORE	AFTER
오늘 TOP 100 응답	풀스캔 + 정렬	ZREVRANGE 단건
"내 등수" 조회	SELECT + 정렬	ZREVRANK O(log N)
14일 Slow Query	측정됨	0건 유지

운영 실증 — Mattermost · Notion · Jenkins · Grafana

실 운영 캡처





서비스

AI 기반 예산 관리·소비 코칭 모바일 앱 🏆 특화 PJT 우수상 1등

이상 소비 탐지·개인화 예산 추천·기프트콘 통합 관리·게이미피케이션으로 건전한 소비 습관 형성 지원

GitHub Repository

기술 스택



역할 — Backend

모바일 OAuth2 SSO

카카오 로그인 + 딥링크(toktokggubi://oauth/kakao) 복귀 + JWT RTR

OCR 파이프라인

Google Cloud Vision → 기프트콘 brand·label·expiresAt 파싱 → 정형 엔티티

실시간 동기화

N빵 계산기 / 물빵 대결 / 흔들기 점수 WebSocket 실시간 동기화

외부 API 통합

SSAFY 금융 OpenAPI / OAuth / OCR — 타임아웃·재시도·예외 격리

도메인 모델링

예산/소비/혜택/카드/기프트콘 핵심 엔티티 + 트랜잭션 설계

작업 기간

2025. 08. 25 - 2025. 09. 26 (5주)

SSAFY 13기 특화 PJT (핀테크 도메인)

구성원

총 6명

FE 3명 (RN/Expo) · BE 2명 (본인 포함) · Infra/AI 1명

프로젝트 회고

· 모바일 딥링크 OAuth2 SSO 직접 설계

- Authorization Code 교환 → 사용자 조회/생성 → JWT 발급 → 앱 딥링크 복귀
- 웹과 다른 모바일 앱 환경의 인증 흐름 끊김 없이 연결

· OCR → 도메인 파싱 파이프라인 구축

- Google Vision 결과(비정형) → 정형 엔티티(brand/label/expires) 변환·저장

· 이기종 외부 API 통합 안정화

- SSAFY 금융 / OAuth / OCR — 각기 다른 지연 시간을 타임아웃·재시도·격리로 흡수

· WebSocket 실시간 동기화에서 연결·재구독 이슈를 사용자 식별 방식으로 해결

PROBLEM → SOLUTION

"스타벅스 / Starbucks / 스타벅스 코리아"가 서로 다른 가맹점 ID로 저장되면 흔들기 후 매장 매칭 실패. Tesseract 한국어 OCR 정확도 한계
→ GCP Vision API + 도메인 텍스트 파서 2단 구조 + 캐노니컬 매핑으로 정형화.

Before vs After

BEFORE

자유 입력 + Tesseract 자체 OCR

- ✗ 브랜드 표기 흔들림 — "스타벅스 / Starbucks / 스타벅스 코리아" 다른 ID 저장
- ✗ 입력 부담 — 한 장당 4 칸 자유 입력 (12장이면 48칸)
- ✗ 만료일 형식 흔들림 — "2025.11.30 / -11-30 / 11월 30일"
- ✗ Tesseract 한국어 정확도 한계 — 폰트·해상도에 민감, 후처리 보강 필요
- ✗ 흔들기 매장 매칭 단계에서 본인 기프트콘 누락 발생

AFTER

GCP Vision API + GifticonTextParser

- ✓ OCR 정확도 — Google Cloud Vision API로 한글 인식을 안정화
- ✓ 도메인 분리 — OCR 결과를 GifticonTextParser에서 정형화
- ✓ 캐노니컬 매핑 — 브랜드 별칭 사전으로 동일 franchiseId 보장
- ✓ 정규식 LocalDate 파싱 — 모든 만료일 형식 통일 변환
- ✓ 이미지 한 장으로 brand·label·expiresAt 자동 등록

2단 분리 파이프라인 — 책임 분리로 정확도 + 유지보수성 확보



domain/gifticon/parser/GifticonTextParser.java

정규식 + 캐노니컬 매핑

```
@Component @RequiredArgsConstructor
public class GifticonTextParser {

    private final FranchiseCanonicalMap canonicalMap;

    // 만료일 정규식 - 다양한 형식 통합 파싱
    private static final Pattern EXPIRES_RE = Pattern.compile(
        "(\\Ww{4})[\\Ww\\.\\Ww-\\/\\n]\\Ww*(\\Ww{1,2})[\\Ww\\.\\Ww-\\/\\n]\\Ww*(\\Ww{1,2})"
    );

    public GifticonParseResult parse(String ocrText) {
        // 1. 브랜드 추출 + 캐노니컬 매핑
        String rawBrand = extractBrand(ocrText);
        Long franchiseId = canonicalMap.resolve(rawBrand);
        // "Starbucks" / "스타벅스" / "스벅" → 동일 ID

        // 2. 만료일 LocalDate 파싱
        LocalDate expiresAt = parseExpiresAt(ocrText);

        // 3. 상품명 - 브랜드 · 만료일 제외 후 추출
        String label = extractLabel(ocrText, rawBrand);

        return new GifticonParseResult(franchiseId, label, expiresAt);
    }

    private LocalDate parseExpiresAt(String text) {
        var m = EXPIRES_RE.matcher(text);
        if (!m.find()) return null;
        return LocalDate.of(
            Integer.parseInt(m.group(1)),
            Integer.parseInt(m.group(2)),
            Integer.parseInt(m.group(3))
        );
    }

    // 캐노니컬 매핑 사전
    public class FranchiseCanonicalMap {
        public Long resolve(String raw) {
            return switch (normalize(raw)) {
                case "스타벅스", "starbucks", "스벅" -> STARBUCKS_ID;
                case "이디야", "ediya" -> EDIYA_ID;
                default -> findBySimilarity(raw);
            };
        }
    }
}
```

자동 등록

사용자 입력 부담 ↓

이미지 1장 → brand·label·expiresAt 동시 추출

2단 구조

OCR / 도메인 분리

GCP Vision은 텍스트만, 정형화는 도메인 책임

동일 ID

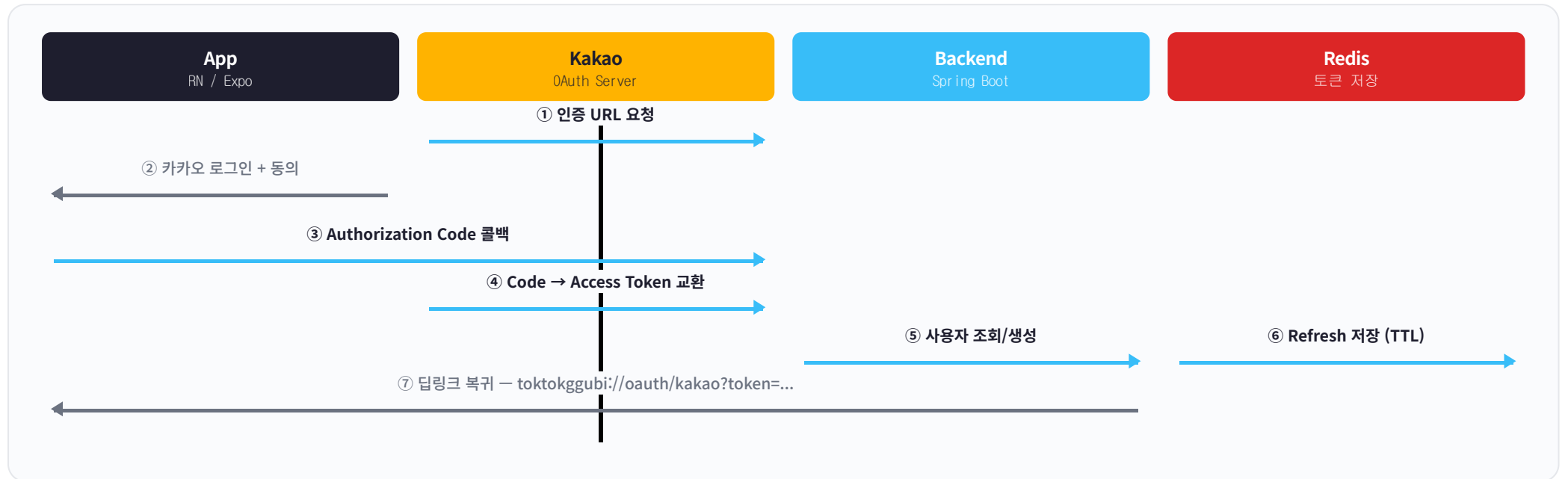
매장 매칭 정확도

캐노니컬 매핑으로 흔들기 결과 매칭 보장

PROBLEM → SOLUTION

웹 OAuth2와 달리 **앱 환경**은 브라우저 → 앱 복귀 흐름이 끊긴다. Authorization Code 교환 후 `toktokggubi://oauth/kakao` **커스텀 스킴 딥링크**로 안전 귀환 + JWT(Access/Refresh) 발급으로 이후 요청 인증.

OAuth2 + 딥링크 시퀀스



domain/auth/controller/AuthController.java

Kakao OAuth2 + 딥링크

```
@RestController @RequiredArgsConstructor
@RequestMapping("/auth/kakao")
public class AuthController {

    private final KakaoAuthService kakaoService;
    private final JwtProvider jwtProvider;

    // ① 앱이 호출 - 카카오 인증 URL 생성
    @GetMapping("/authorize-url")
    public AuthorizeUrlResponse authorizeUrl() {
        return new AuthorizeUrlResponse(kakaoService.buildAuthorizeUrl());
    }

    // ③~⑦ 카카오 콜백 - Code 교환 + 딥링크 복귀
    @GetMapping("/callback")
    public ResponseEntity<Void> callback(@RequestParam String code) {

        // 1. Authorization Code → Access Token 교환
        KakaoTokenResponse kakaoToken = kakaoService.exchangeToken(code);

        // 2. 카카오 프로필 조회 → 회원 조회/생성
        Member member = kakaoService.findOrCreate(kakaoToken);

        // 3. JWT 발급 (Access + Refresh, RTR)
        String accessToken = jwtProvider.issueAccess(member);
        String refreshToken = jwtProvider.issueRefresh(member);

        // 4. Refresh Token Redis 저장 (TTL)
        jwtProvider.saveRefresh(member.getId(), refreshToken);

        // 5. ★ 커스텀 스킴 딥링크로 복귀 - 앱이 자동 캐치
        String deepLink = "toktokggubi://oauth/kakao"
            + "?accessToken=" + accessToken
            + "&refreshToken=" + refreshToken;

        return ResponseEntity.status(HttpStatus.SEE_OTHER)
            .location(URI.create(deepLink))
            .build();
    }

    // Refresh Token Rotation - 탈취 시나리오 방어
    @PostMapping("/token/refresh")
    public TokenResponse refresh(@RequestBody RefreshRequest req) {
        return jwtProvider.rotate(req.refreshToken());
    }
}
```

APP RETURN

커스텀 스킴 딥링크

`toktokggubi://oauth/kakao` 로 앱이 등록한 스킴 사용. 외부 브라우저 → 앱 자동 복귀로 **인증 흐름 단절 없음**.

JWT FLOW

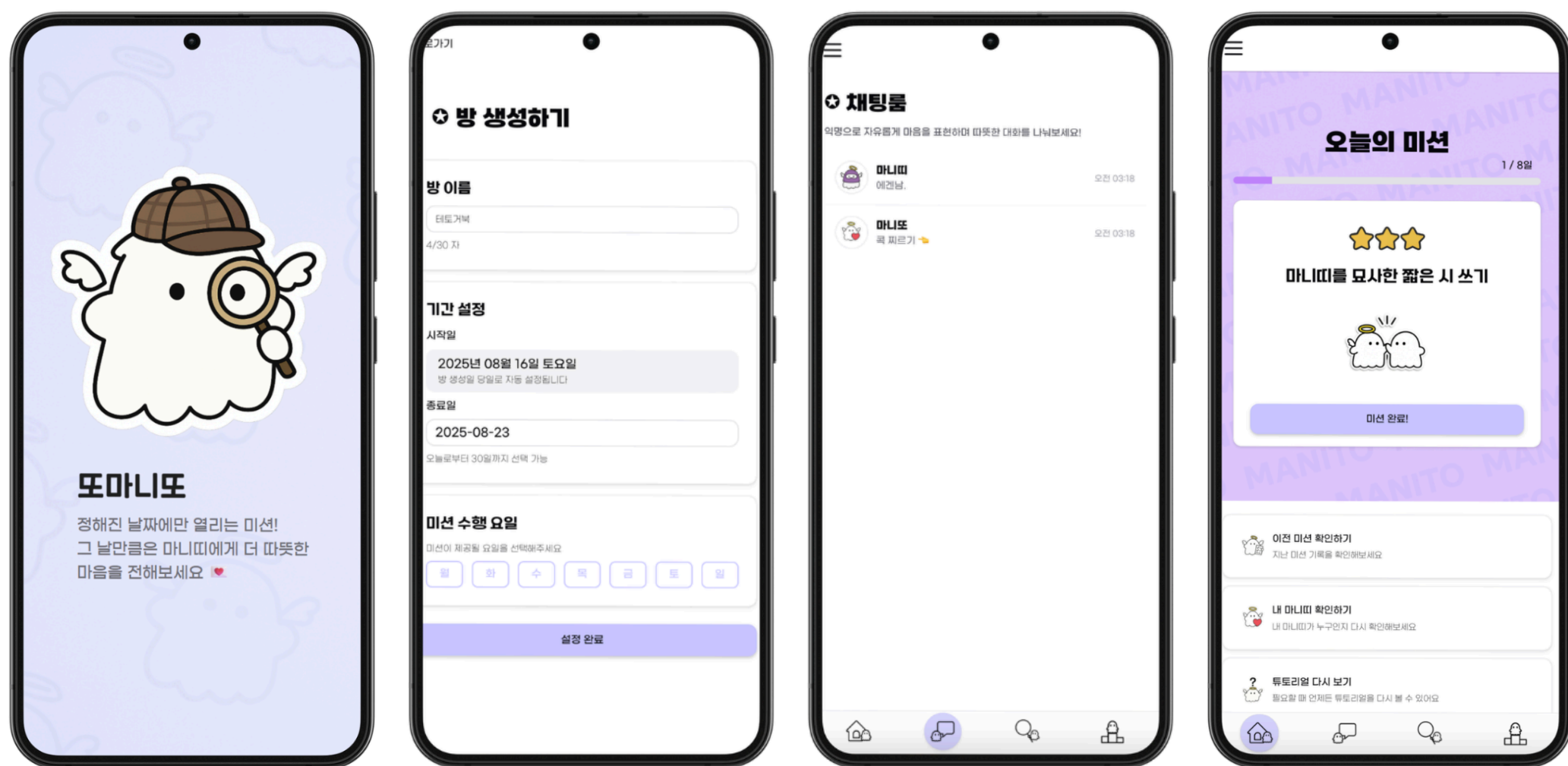
Access · Refresh 분리 + RTR

Access는 짧은 TTL · Refresh는 Redis TTL 보관. Refresh 쓰는 순간 **새 토큰 발급 + 이전 무효화**로 탈취 시나리오 방어.

FILTER + REVOCATION

JwtAuthenticationFilter

요청마다 JWT 검증 + `auth:blacklist:{jti}` 확인. `permitAll` 경로 정책으로 OAuth2 콜백 인증 우회.



서비스

실시간 익명 채팅 마니또 게임 모바일 웹

익명 1:1 채팅·미션·푸시 알림·AI 성향 분석으로 사용자 간 자연스러운 교류를 지원하는 소셜 웹앱

[GitHub Repository](#)

기술 스택



역할 — Backend

실시간 1:1 채팅

WebSocket+STOMP 라우팅 / MongoDB 채팅 메시지 영속

JWT Principal 통일

WebSocket 인증 충돌 → JWT 기반 Principal 주입으로 1:1 라우팅 안정화

인증/보안

JWT + OAuth2 / Redis RTR / Nginx·SSL·CORS 환경 대응

AI 성향 분석

GPT-4.1 mini 연동 — 채팅 참여자 성향 분석 API 제공

PWA 푸시

Firebase FCM — 모바일 웹 실시간 알림 + PWA 설치

작업 기간

2025. 07. 14 - 2025. 08. 15 (5주)

SSAFY 13기 공동 프로젝트 (웹기술)

구성원

총 6명

FE 3명 · BE 2명 (본인 포함) · Infra 1명

프로젝트 회고

- **WebSocket 인증 충돌 → JWT Principal 주입으로 재설계**
 - STOMP 핸드셰이크 + 채널 메시지에서 동일한 사용자 식별 보장
 - Principal.getName() = userId 통일로 1:1 라우팅 안정화
- **3DB 분리 — MySQL(메타) · MongoDB(메시지) · Redis(세션·캐시)**
 - 채팅 영속과 메타 데이터의 갱신 빈도·정렬 비용 차이를 저장소로 분리
- **운영 환경 보안 이슈 해결 — Nginx·SSL·CORS 실서비스 경험**
 - JWT 쿠키 처리 + Refresh Token 회전 + 세션 안정화
- **모바일 웹 PWA 적용으로 앱 체감 향상 + 푸시 알림 자동 수신**

PROBLEM → SOLUTION

마니또 ↔ 마니띠 메시지가 두 사람 사이에서만 오가야 게임이 성립한다. HTTP 인증(`SecurityContext` · `ThreadLocal`)과 WebSocket 채널 (비동기)이 분리되어 `Principal.getName()` 이 비어 1:1 라우팅 실패 → **JwtUserDetails 단일 객체로 통일**해 해결.

Before vs After

BEFORE

HTTP-only Principal — WebSocket 채널에서 미주입

[HTTP 요청]
└─ JwtFilter → SecurityContext.set(auth) ✓
[WebSocket 메시지]
└─ Principal == null ✕
└─ /user/queue/chat.5 라우팅 실패

- ✕ HTTP의 `ThreadLocal` 인증 정보가 비동기 WS 채널에 전파되지 않음
- ✕ `Principal.getName() = null` → `convertAndSendToUser` 무동작
- ✕ 다른 방으로 메시지가 새거나 도착 안 함 → 게임성 붕괴

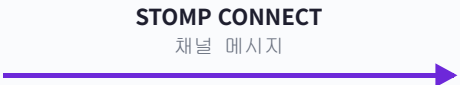
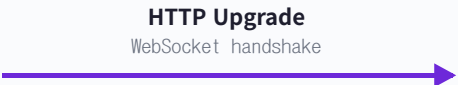
AFTER

JwtUserDetails — UserDetails + Principal 동시 구현

[Handshake] JwtHandshakeInterceptor
└─ JWT 검증 → JwtUserDetails 생성
└─ attributes.put("user", principal)
[STOMP CONNECT] WsAuthChannelInterceptor
└─ SimpUserHeader = JwtUserDetails
└─ Principal.getName() = userId ✓

- ✓ 2단계 인터셉터 — Handshake(HTTP 단계) + ChannelInterceptor(STOMP 단계)
- ✓ 커스텀 Principal — JwtUserDetails가 동시에 인증 객체+사용자 식별자
- ✓ `Principal.getName() = userId` 통일 → 1:1 라우팅 안정화

2단계 인터셉터 — Handshake + Channel



```
@Getter @RequiredArgsConstructor
public class JwtUserDetails
    implements UserDetails, Principal {

    private final Long userId;
    private final String nickname;
    private final Collection<? extends GrantedAuthority> authorities;

    // ★ Principal.getName() == userId - 1:1 라우팅 키
    @Override
    public String getName() {
        return String.valueOf(userId);
    }

    // UserDetails 메서드들 (생략)
    @Override public String getUsername() { return getName(); }
    @Override public String getPassword() { return null; }
}

// JwtHandshakeInterceptor - HTTP 업그레이드 단계
public boolean beforeHandshake(ServerHttpRequest req, ...,
                                Map<String, Object> attributes) {
    String token = extractToken(req);
    JwtUserDetails principal = jwtProvider.parse(token);
    attributes.put("user", principal); // ★ STOMP 단계로 전달
    return true;
}

// WsAuthChannelInterceptor - STOMP CONNECT 단계
@Override
public Message<?> preSend(Message<?> msg, MessageChannel ch) {
    StompHeaderAccessor acc = StompHeaderAccessor.wrap(msg);
    if (StompCommand.CONNECT.equals(acc.getCommand())) {
        JwtUserDetails p = (JwtUserDetails) acc.getSessionAttributes().get("user");
        acc.setUser(p); // ★ Principal.getName() = userId 일관 주입
    }
    return msg;
}
```

1:1 라우팅

메시지 정확도

/user/queue/chat 채널에 정확한 사용자만 도달

단일 객체

JwtUserDetails 통합

UserDetails + Principal 동시 구현으로 HTTP·WS 컨텍스트 통일

2 단계

인터셉터 설계

Handshake (HTTP) + Channel (STOMP) 책임 분리

DESIGN DECISION

"채팅"이라는 단어로 묶이지만 **메타 데이터**·**메시지**·**세션**은 갱신 빈도와 일관성 요구가 다르다. 단일 DB로 끼워 맞추는 대신 **3DB 책임 분리**로 확장성과 성능을 동시 확보.

3DB 분리 — 책임에 따른 저장소 선택

RDB MySQL 메타 데이터 · 강한 일관성 member · room · chatting_room · manito_assignment · mission · fcm_token 외래 키 + 트랜잭션이 필요한 도메인. 매칭 · 인증 · 미션 영속.	NoSQL MongoDB 채팅 메시지 · append-only chat_message { senderUserId, chattingRoomId, type, content, sentAt } 빠른 append + 시간 역순 페이지네이션. 외래 키 없이 문서 단위로 자유롭게 저장.	In-Memory Redis 세션 · 캐시 · 토큰 refresh:{userId} blacklist:{jti} fcm:cache:{userId} 빠른 read/write + TTL. RTR(Refresh Token Rotation) 토큰 회전, 인증 세션 안정화.
---	--	--

이벤트별 DB 라우팅 — 같은 "채팅"이라도 다른 저장소

로그인 JWT 발급	member 조회	—	refresh 저장 (TTL)
방 생성 매칭	room + chatting_room INSERT	—	—
채팅 발송 STOMP	lastMessage 갱신	chat_message append	—
채팅 페이지 역순 조회	—	chat_message find (sentAt DESC)	—
로그아웃 토큰 무효	—	—	blacklist 등록 + refresh 삭제

A AI 성향 분석 (GPT-4.1 mini)

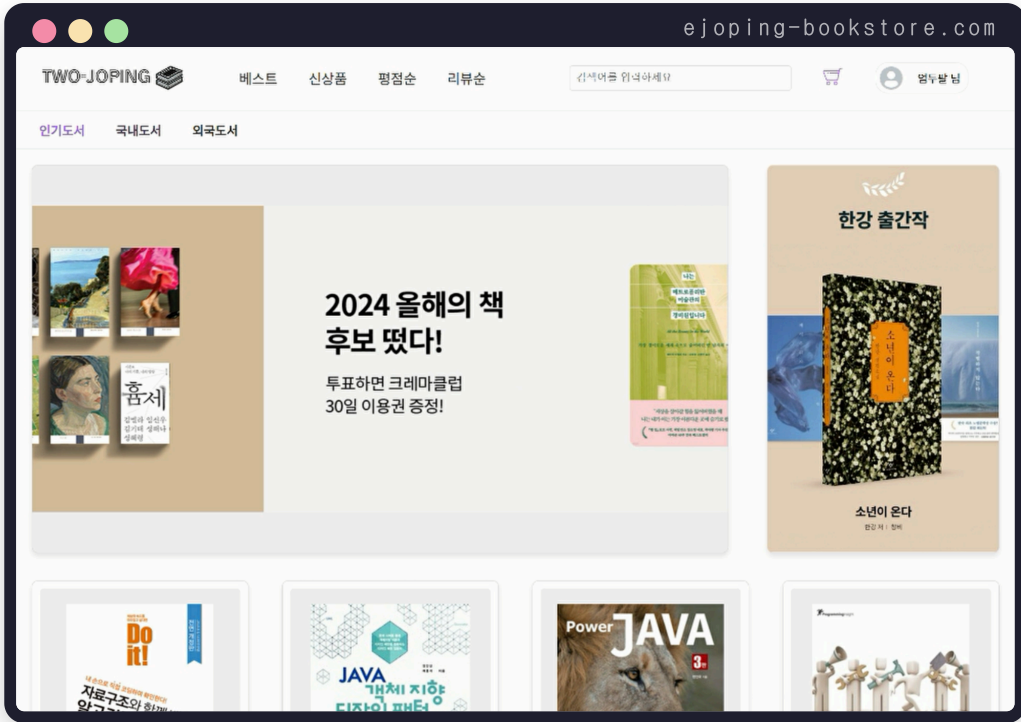
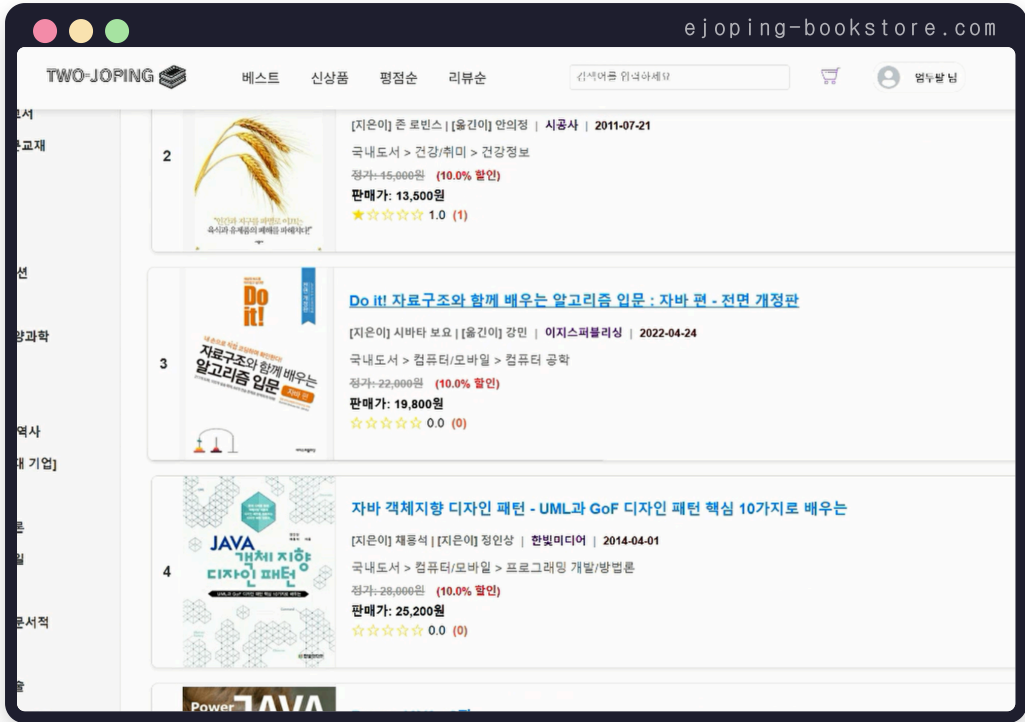
[입력] 마니띠 채팅 메시지 30건
[프롬프트] "이 사용자의 성향 Top 3"
[모델] gpt-4.1-mini (OpenAI)
↓
[출력] { friendly, curious, careful }
↓ MongoDB analysis 컬렉션 저장

- 비식별화 — 메시지 내용만 추출, userId 제외하여 프롬프트 구성
- 호출 제한 — 일일 1회 + 캐시로 토큰 비용 통제
- 마니띠 본인에게만 분석 결과 노출 (권한 검증)

B Firebase FCM PWA 푸시

[웹앱] Service Worker 등록 + 토큰 발급
↓ MySQL fcm_token 저장
[이벤트] 새 메시지 / 미션 도착
↓ FCM API 호출
[디바이스] 백그라운드 푸시 알림
↓ 앱 미설치 상태에서도 수신

- PWA 설치 — 모바일 웹을 앱처럼 홈 화면 추가
- 오프라인 대응 — Service Worker 캐싱으로 빠른 재진입
- SSL 필수 — 운영 Nginx + Let's Encrypt 인증서 적용



서비스

온라인 도서 쇼핑몰 (대용량 커머스 플랫폼)

Spring Boot + Thymeleaf 기반 도서 커머스 / Elasticsearch + Nori 형태소 분석기로 검색 엔진 도입

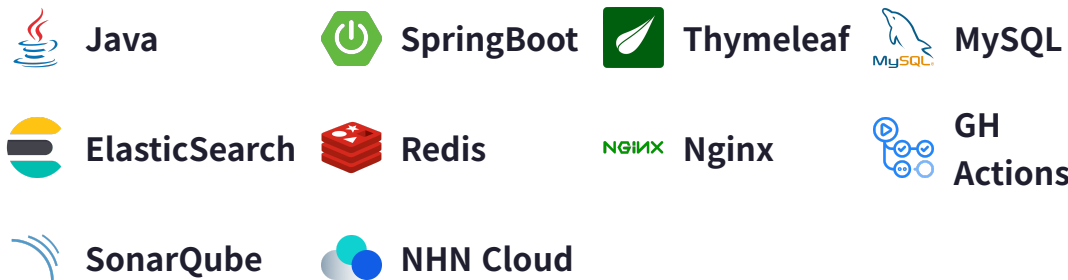
[GitHub Repository](#)

작업 기간

2024. 10. 14 - 2024. 12. 13 (9주)

NHN Academy 7기 온라인 도서 쇼핑몰 프로젝트

기술 스택



구성원

총 8명

FullStack 6명 (본인 포함) · Infra 2명

역할 — FullStack

Backend

검색 엔진 도입

MySQL → Elasticsearch 마이그레이션 / nori + ngram + jaso 분석기 설계

도메인 API 설계·구현

ERD 설계 + Spring Boot REST API (카테고리·검색·기여자·배송)

Frontend

Thymeleaf layout

화면 구성 + 데이터 출력 + 담당 도메인 페이지 퍼블리싱

협업 표준화

대규모 팀(8인) 일정 관리

WBS 기반 분담 + API 기술 문서화 + 스프린트 회고 주도

프로젝트 회고

- **RDBMS 검색 한계 분석** — MySQL FULLTEXT 평균 1.2초 응답 / Slow Query 빈발
 - JOIN N+1 + ORDER BY 조합으로 LIKE 검색의 병목 체감
- **ElasticSearch 도입으로 구조 개선** — 검색 전용 인덱스 분리 구축
 - 한글 분석기 nori + n-gram + jaso 조합 설계
 - 복합 검색어 환경에서도 응답 약 **280ms (76% 단축)**
- **형태소 분석으로 검색 본질 학습**
 - "검색은 LIKE 가 아니다" — 인덱스 매핑·재색인·동의어 처리 경험
- **대규모 팀(8인) 협업 리드**
 - WBS · 코드 리뷰 · 스프린트 회고 주도로 생산성 관리

PROBLEM → SOLUTION

개발 단계 검색 응답 평균 **1,200ms** · Slow Query **30%** 발생. **LIKE '%키워드%'** 앞 와일드카드는 B-Tree 인덱스 사용 불가 → MySQL FULLTEXT 한계 확인 후 **Elasticsearch + 한글 형태소 분석기** 도입 + **Logstash JDBC** 로 10초 단위 자동 동기화.

Before vs After

BEFORE

QueryDSL .contains() + 4 LEFT JOIN

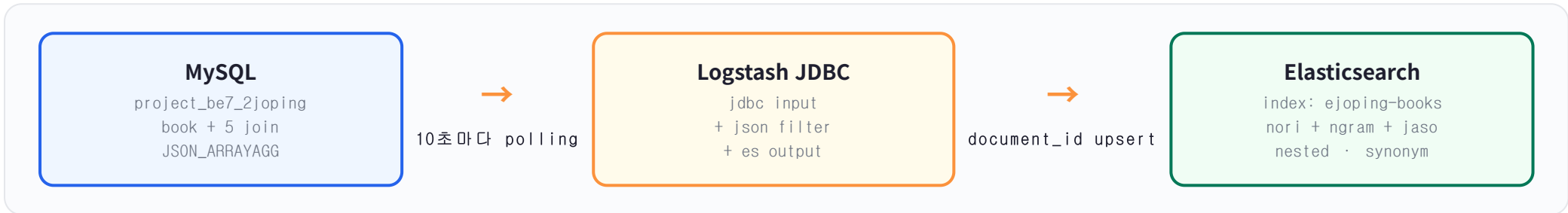
- ✗ Full Table Scan — LIKE '%keyword%' 앞 와일드카드로 인덱스 미사용
- ✗ 4 테이블 LEFT JOIN — contributor·tag·category 결과 행 폭발
- ✗ N+1 쿼리 — 도서별 기여자/카테고리 추가 조회
- ✗ 한국어 형태소 미지원 — '자바' 검색 시 '자바스크립트' 누락
- ✗ 관련성 정렬 미지원 — 단순 views 컬럼 정렬만 가능

AFTER

Elasticsearch + nori 형태소 분석기

- ✓ 역색인 + 형태소 토큰화 — Inverted Index 기반 빠른 검색
- ✓ multi_match + nested — 본문/카테고리/태그/기여자 동시 검색
- ✓ 관련성 점수(BM25) + 가중치 — title^10 / desc^3 / publisher^2
- ✓ 한글 분석기 nori + ngram + jaso — 부분 검색 · 자모 검색 지원
- ✓ Logstash 자동 동기화 — MySQL → ES 10초 polling upsert

인덱스 운영 — MySQL → Logstash → Elasticsearch



검색 쿼리 — multi_match + nested 결합

```
bookset/elastic/repository/impl/ElasticRepositoryImpl.java multi_match · nested · BM25

public Page<ElasticSearchResponseDto> searchByKeyword(String keyword, Long categoryId, Pageable pageable, String sort) {
    var req = new SearchRequest.Builder()
        .index(elasticConfig.getBookIndexName()) // "ejoping-books"
        .from((int) pageable.getOffset())
        .size(pageable.getPageSize())
        .query(q -> q.bool(b -> {

            // ① 본문 필드 multi_match - 가중치 + 4종 분석기 활용
            b.should(s -> s.multiMatch(m -> m.query(keyword).fields(
                "book_title^10", "book_title.ngram^3",
                "book_title.jaso", "book_title.synonym",
                "book_desc^3", "book_desc.ngram", "book_desc.synonym",
                "book_publisher^2", "book_publisher.ngram", "book_publisher.synonym"
            ))));

            // ② 카테고리 nested - 트리 구조 책 분류 검색
            b.should(s -> s.nested(n -> n.path("book_category").query(
                cq -> cq.multiMatch(m -> m.query(keyword).fields(
                    "book_category.name^3", "book_category.name.ngram", "book_category.name.synonym"
                ))));

            // ③ 태그 nested · ④ 기여자 nested (저자/번역/출판) 동일 패턴
            // b.should(...book_tag...); b.should(...book_contributor...);

            return b;
        }));

    // 정렬 - popularity / newest / lowest_price / highest_price / rating / reviews
    sortCriteria.forEach(e -> req.sort(s -> s.field(f -> f.field(e.getKey()).order(...))));

    // → 평균 280ms · Slow Query 0건
    return elasticsearchClient.search(req.build(), Map.class);
}
```

76% ↓

평균 응답 시간

1,200ms → 280ms

0건

Slow Query

이전 30% → 0건

10초

Logstash 동기화

MySQL → ES 자동 upsert

1 쿼리

검색 호출

4 JOIN → multi_match + nested

한글 검색 3 분석기 조합

ejoping-books index

①

my_nori_analyzer

한국어 형태소 분석

```
tokenizer: nori_tokenizer
decompound_mode: "mixed" filter: ·
nori_part_of_speech (불용 품사 제거
E·J·MM···) · synonym_filter (자바=java)
· lowercase · trim · stop char_filter:
html_strip
```

"객체지향" → [객체, 지향, 객체지향] 형태소 분리. 불용 품사·동의어·HTML 태그 까지 정제하여 본문 검색 정확도 향상.

②

ngram_analyzer

부분 일치 / 자동완성

```
tokenizer: ngram_tokenizer min_gram: 2
max_gram: 3 filter: · 동일 (lowercase,
stop, nori_part_of_speech, synonym_filter)
```

"자바스크립트" → [자바, 바스, 스크, 크림, 립트, ...]
2~3자 토큰 생성. 중간 일부만 맞아도 인덱스 매칭.

③

suggest_(index/search)

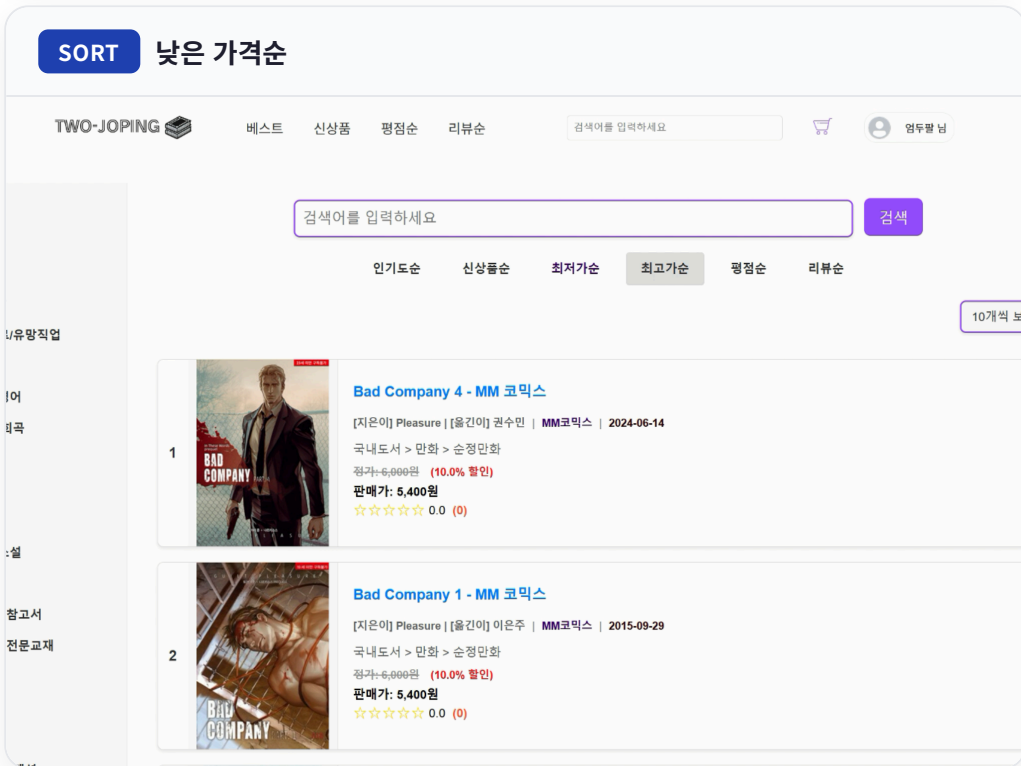
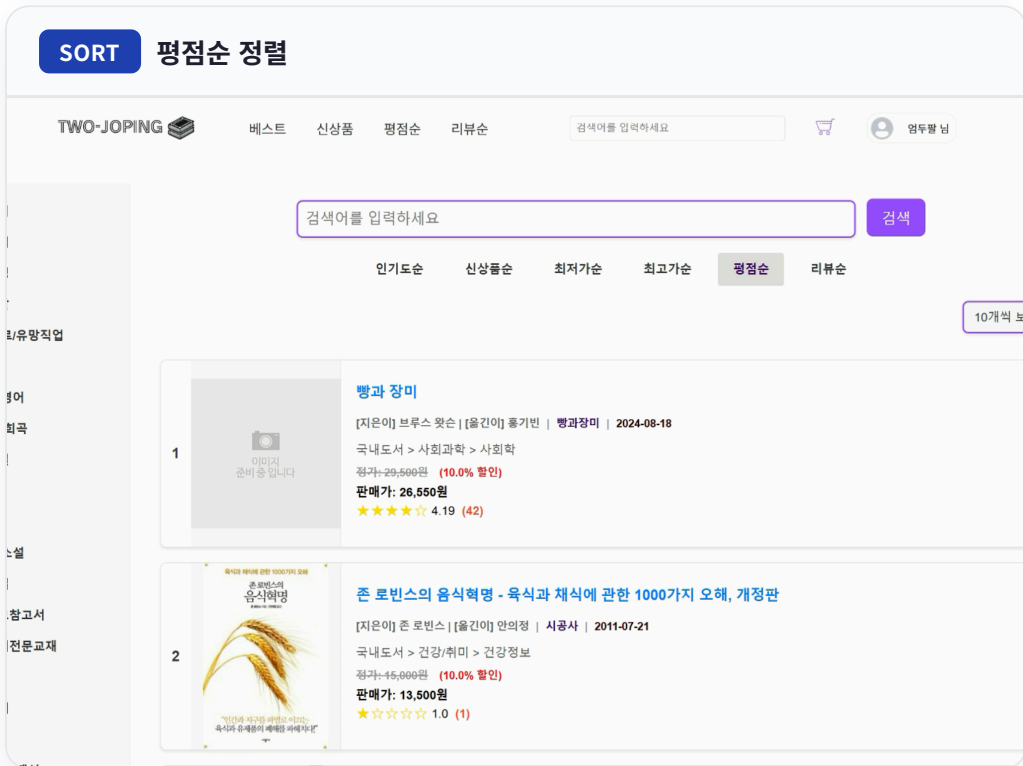
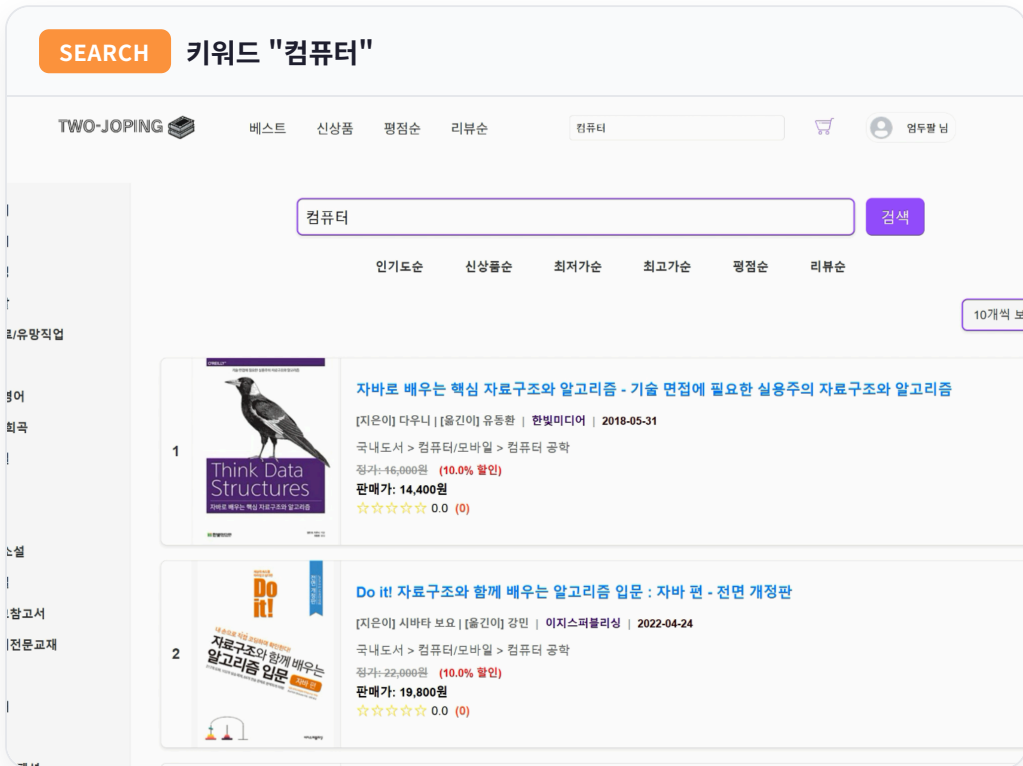
자모 검색 / 오타 허용

```
tokenizer: jaso_tokenizer mistype: true
· 오타 허용 chosung: index true / search
false filter (인덱스만): · edge_ngram
(1-50) suggest_filter
```

한글을 자모 분해. 색인엔 초성 포함으로 "자바스크립트" 같은 초성 검색 + 오타 보정 자동완성 지원.

검색 결과 화면 — 분석기 + 가중치 + 정렬 6 종

ejoping-bookstore.com



A

검색 인덱스 운영

- ▶ 운영 DB와 검색 전용 인덱스 분리 — 부하 격리
- ▶ Logstash JDBC input — MySQL 5 join + JSON_ARRAYAGG → ES upsert
- ▶ 정렬 6종: popularity · newest · price · rating · reviews
- ▶ BM25 + 가중치 — title^10 · desc^3 · publisher^2

B

8인 팀 협업 표준화

- ▶ WBS 기반 일정 관리 + 분담 및 마일스톤 추적
- ▶ API 기술 문서화 표준화 (Swagger + 명세서)
- ▶ 스프린트 회고 주도 — 매 스프린트 KPT 작성
- ▶ Pull Request 코드 리뷰 정책 + SonarQube 정적 분석